

- Description

commons-math-4.0-beta1-src\commons-math-legacy\src\main\java\org\apache\commons\math4\legacy\analysis\interpolation\BicubicInterpolatingFunction.java

- Specification

module BicubicInterpolatingFunction

type

Nx, Ny: int

X, Y: real

Z: real

i, j: int

tx, ty: real

ex: set of {

None,

NullPointerException,

NoDataException,

NumbersTooSmallException,

DimensionMismatchException,

NonMonotonicSequenceException,

NotFiniteNumberException,

OutOfRangeException

}

process Construct(x, y, f, fx, fy, fxy) result: (ex: set)

Functional Scenario Form

x = 0

&& ex = NullPointerException ||

length = 0

&& ex = NoDataException ||

length < 2

&& ex = NumbersTooSmallException ||

rows=0

&& ex = DimensionMismatchException ||

x < k+1

&& ex = NonMonotonicSequenceException ||

y < k+1

&& ex = NonMonotonicSequenceException ||

isFinite=0

&& ex = NotFiniteNumberException ||

isFinitey=0

&& ex = NotFiniteNumberException ||

isFinite=1 && ex = None

end_process

process LocateCell(X, Y) result: (i: int, j: int, tx: real, ty: real, ex: set)

Functional Scenario Form

(not isFinite(X) or not isFinite(Y))
 && (ex = NotFiniteNumberException) or
(X < x[0] or X > x[Nx-1] or Y < y[0] or Y > y[Ny-1])
 && (ex = OutOfRangeException) or
(X = x[Nx-1] and Nx ≥ 2) && (i = Nx-2; 0 ≤ j ≤ Ny-2; tx = 1.0; ty ∈ [0,1]; ex = None) or
(Y = y[Ny-1] and Ny ≥ 2) && (j = Ny-2; 0 ≤ i ≤ Nx-2; ty = 1.0; tx ∈ [0,1]; ex = None) or
(x[i] ≤ X < x[i+1] and y[j] ≤ Y < y[j+1] for some i ∈ [0..Nx-2], j ∈ [0..Ny-2])
 && (tx = (X - x[i]) / (x[i+1] - x[i]);
 ty = (Y - y[j]) / (y[j+1] - y[j]); ex = None)

end_process

process Value(X, Y) result: (Z: real, ex: set)

Functional Scenario Form

(LocateCell(X, Y) yields ex ≠ None) && (ex = that ex) or

(LocateCell ok and all 16 finite:
 F00=f[i][j], F10=f[i+1][j], F01=f[i][j+1], F11=f[i+1][j+1],
 Fx00=fx[i][j], Fx10=fx[i+1][j], Fx01=fx[i][j+1], Fx11=fx[i+1][j+1],
 Fy00=fy[i][j], Fy10=fy[i+1][j], Fy01=fy[i][j+1], Fy11=fy[i+1][j+1],
 Fxy00=fxy[i][j],...,Fxy11=fxy[i+1][j+1])
 && (ex = None
 && Z = BicubicHermite(tx,ty; F**,Fx**,Fy**,Fxy**)
) or
(LocateCell ok and any of 16 values non-finite) && (ex = NotFiniteNumberException)

end_process

process PartialX(X, Y) result: (Zx: real, ex: set)

Functional Scenario Form

ex = None && Zx = $\partial/\partial x$ of BicubicHermite at (tx,ty) scaled by $\Delta x = x[i+1]-x[i]$

end_process

process PartialY(X, Y) result: (Zy: real, ex: set)

Functional Scenario Form

ex = None && Zy = $\partial/\partial y$ of BicubicHermite at (tx,ty) scaled by $\Delta y = y[j+1]-y[j]$

end_process

end_module

- Java Code

```
/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. See the NOTICE file distributed with
 * this work for additional information regarding copyright ownership.
 * The ASF licenses this file to You under the Apache License, Version
2.0
 * (the "License"); you may not use this file except in compliance with
 * the License. You may obtain a copy of the License at
 *
 *      http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or impl
ied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
package org.apache.commons.math4.legacy.analysis.interpolation;
import java.util.Arrays;
import org.apache.commons.math4.legacy.exception.NullArgumentException;
import org.apache.commons.math4.legacy.exception.NoDataException;
import org.apache.commons.math4.legacy.exception.NumberIsTooSmallExcept
ion;
import org.apache.commons.math4.legacy.exception.DimensionMismatchExcep
tion;
import org.apache.commons.math4.legacy.exception.NonMonotonicSequenceEx
ception;
import org.apache.commons.math4.legacy.exception.NotFiniteNumberExcepti
on;
import org.apache.commons.math4.legacy.exception.OutOfRangeException;
import java.util.function.DoubleBinaryOperator;
import java.util.function.Function;
import org.apache.commons.numbers.core.Sum;
import org.apache.commons.math4.legacy.analysis.BivariateFunction;
import org.apache.commons.math4.legacy.exception.DimensionMismatchExcep
tion;
import org.apache.commons.math4.legacy.exception.NoDataException;
import org.apache.commons.math4.legacy.exception.NonMonotonicSequenceEx
ception;
import org.apache.commons.math4.legacy.exception.OutOfRangeException;
```

```

import org.apache.commons.math4.legacy.core.MathArrays;
import java.util.Locale;
/**
 * Function that implements the
 * <a href="http://en.wikipedia.org/wiki/Bicubic_interpolation"> bicubic spline
 * interpolation</a>.
 *
 * @since 3.4
 */
public class BicubicInterpolatingFunction implements BivariateFunction
{
    /** Number of coefficients. */
    private static final int NUM_COEFF = 16;
    /**
     * Matrix to compute the spline coefficients from the function values
     and
     * function derivatives values.
     */
    private static final double[][] AINV = { { 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0 },
        { 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 }, { -3, 3, 0, 0, -
2, -1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
        { 2, -
2, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 }, { 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, 0, 0, 0, 0, 0, 0 },
        { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0 }, { 0, 0, 0, 0, 0,
0, 0, 0, 0, -3, 3, 0, 0, -2, -1, 0, 0 },
        { 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, -2, 0, 0, 1, 1, 0, 0 }, { -
3, 0, 3, 0, 0, 0, 0, 0, 0, -2, 0, -1, 0, 0, 0, 0, 0 },
        { 0, 0, 0, 0, -3, 0, 3, 0, 0, 0, 0, 0, 0, -2, 0, -1, 0 },
        { 9, -9, -9, 9, 6, 3, -6, -3, 6, -6, 3, -3, 4, 2, 2, 1 },
        { -6, 6, 6, -6, -3, -3, 3, 3, -4, 4, -2, 2, -2, -2, -1, -1 },
        { 2, 0, -
2, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0 }, { 0, 0, 0, 0, 2, 0, -
2, 0, 0, 0, 0, 0, 1, 0, 1, 0 },
        { -6, 6, 6, -6, -4, -2, 4, 2, -3, 3, -3, 3, -2, -1, -2, -1 },
        { 4, -4, -4, 4, 2, 2, -2, -2, 2, -2, 2, -2, 1, 1, 1, 1 } };
    /** Samples x-coordinates. */
    private final double[] xval;
    /** Samples y-coordinates. */
    private final double[] yval;
    /** Set of cubic splines patching the whole data grid. */
    private final BicubicFunction[][] splines;

```

```

/**
 * @param x      Sample values of the x-
coordinate, in increasing order.
 * @param y      Sample values of the y-
coordinate, in increasing order.
 * @param f      Values of the function on every grid point.
 * @param dFdX   Values of the partial derivative of function with res
pect to x
 *              on every grid point.
 * @param dFdY   Values of the partial derivative of function with res
pect to y
 *              on every grid point.
 * @param d2FdXdY Values of the cross partial derivative of function o
n every
 *              grid point.
 * @throws DimensionMismatchException if the various arrays do not c
ontain
 *              the expected number of elements.
 * @throws NonMonotonicSequenceException if {@code x} or {@code y} are
not
 *              strictly increasing.
 * @throws NoDataException if any of the arrays has zero l
ength.
 */
public BicubicInterpolatingFunction(double[] x, double[] y, double[][]
f, double[][] dFdX, double[][] dFdY,
double[][] d2FdXdY) throws DimensionMismatchException, NoDataExcepti
on, NonMonotonicSequenceException {
    this(x, y, f, dFdX, dFdY, d2FdXdY, false);
}
/**
 * @param x      Sample values of the x-
coordinate, in increasing
 *              order.
 * @param y      Sample values of the y-
coordinate, in increasing
 *              order.
 * @param f      Values of the function on every grid poi
nt.
 * @param dFdX   Values of the partial derivative of func
tion
 *              with respect to x on every grid point.
 * @param dFdY   Values of the partial derivative of func
tion

```

```

*                                with respect to y on every grid point.
* @param d2FdXdY                Values of the cross partial derivative o
f
*                                function on every grid point.
* @param initializeDerivatives Whether to initialize the internal dat
a needed
*                                for calling any of the methods that compu
te the
*                                partial derivatives this function.
* @throws DimensionMismatchException if the various arrays do not c
ontain
*                                the expected number of elements.
* @throws NonMonotonicSequenceException if {@code x} or {@code y} are
not
*                                strictly increasing.
* @throws NoDataException        if any of the arrays has zero l
ength.
*/
public BicubicInterpolatingFunction(double[] x, double[] y, double[][]
f, double[][] dFdX, double[][] dFdY,
    double[][] d2FdXdY, boolean initializeDerivatives)
    throws DimensionMismatchException, NoDataException, NonMonotonicSequ
enceException {
    final int xLen = x.length;
    final int yLen = y.length;
    if (xLen == 0 || yLen == 0 || f.length == 0 || f[0].length == 0) {
        throw new NoDataException();
    }
    if (xLen != f.length) {
        throw new DimensionMismatchException(xLen, f.length);
    }
    if (xLen != dFdX.length) {
        throw new DimensionMismatchException(xLen, dFdX.length);
    }
    if (xLen != dFdY.length) {
        throw new DimensionMismatchException(xLen, dFdY.length);
    }
    if (xLen != d2FdXdY.length) {
        throw new DimensionMismatchException(xLen, d2FdXdY.length);
    }
    MathArrays.checkOrder(x);
    MathArrays.checkOrder(y);
    xval = x.clone();
    yval = y.clone();

```

```

final int lastI = xLen - 1;
final int lastJ = yLen - 1;
splines = new BicubicFunction[lastI][lastJ];
for (int i = 0; i < lastI; i++) {
    if (f[i].length != yLen) {
        throw new DimensionMismatchException(f[i].length, yLen);
    }
    if (dFdX[i].length != yLen) {
        throw new DimensionMismatchException(dFdX[i].length, yLen);
    }
    if (dFdY[i].length != yLen) {
        throw new DimensionMismatchException(dFdY[i].length, yLen);
    }
    if (d2FdXdY[i].length != yLen) {
        throw new DimensionMismatchException(d2FdXdY[i].length, yLen);
    }
    final int ip1 = i + 1;
    final double xR = xval[ip1] - xval[i];
    for (int j = 0; j < lastJ; j++) {
        final int jp1 = j + 1;
        final double yR = yval[jp1] - yval[j];
        final double xRyR = xR * yR;
        final double[] beta = new double[] { f[i][j], f[ip1][j], f[i][jp1],
f[ip1][jp1], dFdX[i][j] * xR,
        dFdX[ip1][j] * xR, dFdX[i][jp1] * xR, dFdX[ip1][jp1] * xR, dFdY[i]
[j] * yR, dFdY[ip1][j] * yR,
        dFdY[i][jp1] * yR, dFdY[ip1][jp1] * yR, d2FdXdY[i][j] * xRyR, d2F
dXdY[ip1][j] * xRyR,
        d2FdXdY[i][jp1] * xRyR, d2FdXdY[ip1][jp1] * xRyR };
        splines[i][j] = new BicubicFunction(computeSplineCoefficients(beta)
, xR, yR, initializeDerivatives);
    }
}
}
/**
 * {@inheritDoc}
 */
@Override
public double value(double x, double y) throws OutOfRangeException {
    final int i = searchIndex(x, xval);
    final int j = searchIndex(y, yval);
    final double xN = (x - xval[i]) / (xval[i + 1] - xval[i]);
    final double yN = (y - yval[j]) / (yval[j + 1] - yval[j]);
    return splines[i][j].value(xN, yN);
}

```

```

}
/**
 * Indicates whether a point is within the interpolation range.
 *
 * @param x First coordinate.
 * @param y Second coordinate.
 * @return {@code true} if (x, y) is a valid point.
 */
public boolean isValidPoint(double x, double y) {
    return !(x < xval[0] || x > xval[xval.length - 1] || y < yval[0] || y
> yval[yval.length - 1]);
}
/**
 * @return the first partial derivative respect to x.
 * @throws NullPointerException if the internal data were not initiali
zed (cf.
 *
 *                                     {@link #BicubicInterpolatingFunction(doub
le[],double[],double[][], double[][],double[][],double[][],boolean)
 *                                     constructor}).
 */
public DoubleBinaryOperator partialDerivativeX() {
    return partialDerivative(BicubicFunction::partialDerivativeX);
}
/**
 * @return the first partial derivative respect to y.
 * @throws NullPointerException if the internal data were not initiali
zed (cf.
 *
 *                                     {@link #BicubicInterpolatingFunction(doub
le[],double[],double[][], double[][],double[][],double[][],boolean)
 *                                     constructor}).
 */
public DoubleBinaryOperator partialDerivativeY() {
    return partialDerivative(BicubicFunction::partialDerivativeY);
}
/**
 * @return the second partial derivative respect to x.
 * @throws NullPointerException if the internal data were not initiali
zed (cf.
 *
 *                                     {@link #BicubicInterpolatingFunction(doub
le[],double[],double[][], double[][],double[][],double[][],boolean)
 *                                     constructor}).
 */
public DoubleBinaryOperator partialDerivativeXX() {
    return partialDerivative(BicubicFunction::partialDerivativeXX);
}

```

```

}
/**
 * @return the second partial derivative respect to y.
 * @throws NullPointerException if the internal data were not initiali
zed (cf.
 *
 *                                     {@link #BicubicInterpolatingFunction(doub
le[],double[],double[][], double[][],double[][],double[][],boolean)
 *                                     constructor}).
 */
public DoubleBinaryOperator partialDerivativeYY() {
    return partialDerivative(BicubicFunction::partialDerivativeYY);
}
/**
 * @return the second partial cross derivative.
 * @throws NullPointerException if the internal data were not initiali
zed (cf.
 *
 *                                     {@link #BicubicInterpolatingFunction(doub
le[],double[],double[][], double[][],double[][],double[][],boolean)
 *                                     constructor}).
 */
public DoubleBinaryOperator partialDerivativeXY() {
    return partialDerivative(BicubicFunction::partialDerivativeXY);
}
/**
 * @param which derivative function to apply.
 * @return the selected partial derivative.
 * @throws NullPointerException if the internal data were not initiali
zed (cf.
 *
 *                                     {@link #BicubicInterpolatingFunction(doub
le[],double[],double[][], double[][],double[][],double[][],boolean)
 *                                     constructor}).
 */
private DoubleBinaryOperator partialDerivative(Function<BicubicFunctio
n, BivariateFunction> which) {
    return (x, y) -> {
        final int i = searchIndex(x, xval);
        final int j = searchIndex(y, yval);
        final double xN = (x - xval[i]) / (xval[i + 1] - xval[i]);
        final double yN = (y - yval[j]) / (yval[j + 1] - yval[j]);
        return which.apply(splines[i][j]).value(xN, yN);
    };
}
/**
 * @param c    Coordinate.

```

```

* @param val Coordinate samples.
* @return the index in {@code val} corresponding to the interval containing
aining
*      {@code c}.
* @throws OutOfRangeException if {@code c} is out of the range defined by the
*      boundary values of {@code val}.
*/
private static int searchIndex(double c, double[] val) {
    final int r = Arrays.binarySearch(val, c);
    if (r == -1 || r == -val.length - 1) {
        throw new OutOfRangeException(c, val[0], val[val.length - 1]);
    }
    if (r < 0) {
        // "c" is within an interpolation sub-interval: Return the
        // index of the sample at the lower end of the sub-interval.
        return -r - 2;
    }
    final int last = val.length - 1;
    if (r == last) {
        // "c" is the last sample of the range: Return the index
        // of the sample at the lower end of the last sub-interval.
        return last - 1;
    }
    // "c" is another sample point.
    return r;
}
/**
 * Compute the spline coefficients from the list of function values and
 * function
 * partial derivatives values at the four corners of a grid element. They must
 * be specified in the following order:
 * <ul>
 * <li>f(0,0)</li>
 * <li>f(1,0)</li>
 * <li>f(0,1)</li>
 * <li>f(1,1)</li>
 * <li>f<sub>x</sub>(0,0)</li>
 * <li>f<sub>x</sub>(1,0)</li>
 * <li>f<sub>x</sub>(0,1)</li>
 * <li>f<sub>x</sub>(1,1)</li>
 * <li>f<sub>y</sub>(0,0)</li>
 * <li>f<sub>y</sub>(1,0)</li>

```

```

* <li> $f_{y(0,1)}$ </li>
* <li> $f_{y(1,1)}$ </li>
* <li> $f_{xy(0,0)}$ </li>
* <li> $f_{xy(1,0)}$ </li>
* <li> $f_{xy(0,1)}$ </li>
* <li> $f_{xy(1,1)}$ </li>
* </ul>
* where the subscripts indicate the partial derivative with respect to
the
* corresponding variable(s).
*
* @param beta List of function values and function partial derivative
s values.
* @return the spline coefficients.
*/
private static double[] computeSplineCoefficients(double[] beta) {
    final double[] a = new double[NUM_COEFF];
    for (int i = 0; i < NUM_COEFF; i++) {
        double result = 0;
        final double[] row = AINV[i];
        for (int j = 0; j < NUM_COEFF; j++) {
            result += row[j] * beta[j];
        }
        a[i] = result;
    }
    return a;
}
}
/**
 * Bicubic function.
 */
class BicubicFunction implements BivariateFunction {
    /** Number of points. */
    private static final short N = 4;
    /** Coefficients. */
    private final double[][] a;
    /** First partial derivative along x. */
    private final BivariateFunction partialDerivativeX;
    /** First partial derivative along y. */
    private final BivariateFunction partialDerivativeY;
    /** Second partial derivative along x. */
    private final BivariateFunction partialDerivativeXX;
    /** Second partial derivative along y. */
    private final BivariateFunction partialDerivativeYY;

```

```

/** Second crossed partial derivative. */
private final BivariateFunction partialDerivativeXY;
/**
    * Simple constructor.
    *
    * @param coeff          Spline coefficients.
    * @param xR             x spacing.
    * @param yR             y spacing.
    * @param initializeDerivatives Whether to initialize the internal data
    a needed
    *                          for calling any of the methods that compute the
    *                          partial derivatives this function.
    */
    BicubicFunction(double[] coeff, double xR, double yR, boolean initializeDerivatives) {
        a = new double[N][N];
        for (int j = 0; j < N; j++) {
            final double[] aJ = a[j];
            for (int i = 0; i < N; i++) {
                aJ[i] = coeff[i * N + j];
            }
        }
        if (initializeDerivatives) {
            // Compute all partial derivatives functions.
            final double[][] aX = new double[N][N];
            final double[][] aY = new double[N][N];
            final double[][] aXX = new double[N][N];
            final double[][] aYY = new double[N][N];
            final double[][] aXY = new double[N][N];
            for (int i = 0; i < N; i++) {
                for (int j = 0; j < N; j++) {
                    final double c = a[i][j];
                    aX[i][j] = i * c;
                    aY[i][j] = j * c;
                    aXX[i][j] = (i - 1) * aX[i][j];
                    aYY[i][j] = (j - 1) * aY[i][j];
                    aXY[i][j] = j * aX[i][j];
                }
            }
        }
        partialDerivativeX = (double x, double y) -> {
            final double x2 = x * x;
            final double[] pX = { 0, 1, x, x2 };
            final double y2 = y * y;

```

```

    final double y3 = y2 * y;
    final double[] pY = { 1, y, y2, y3 };
    return apply(pX, 1, pY, 0, aX) / xR;
};

partialDerivativeY = (double x, double y) -> {
    final double x2 = x * x;
    final double x3 = x2 * x;
    final double[] pX = { 1, x, x2, x3 };
    final double y2 = y * y;
    final double[] pY = { 0, 1, y, y2 };
    return apply(pX, 0, pY, 1, aY) / yR;
};

partialDerivativeXX = (double x, double y) -> {
    final double[] pX = { 0, 0, 1, x };
    final double y2 = y * y;
    final double y3 = y2 * y;
    final double[] pY = { 1, y, y2, y3 };
    return apply(pX, 2, pY, 0, aXX) / (xR * xR);
};

partialDerivativeYY = (double x, double y) -> {
    final double x2 = x * x;
    final double x3 = x2 * x;
    final double[] pX = { 1, x, x2, x3 };
    final double[] pY = { 0, 0, 1, y };
    return apply(pX, 0, pY, 2, aYY) / (yR * yR);
};

partialDerivativeXY = (double x, double y) -> {
    final double x2 = x * x;
    final double[] pX = { 0, 1, x, x2 };
    final double y2 = y * y;
    final double[] pY = { 0, 1, y, y2 };
    return apply(pX, 1, pY, 1, aXY) / (xR * yR);
};
} else {
    partialDerivativeX = null;
    partialDerivativeY = null;
    partialDerivativeXX = null;
    partialDerivativeYY = null;
    partialDerivativeXY = null;
}
}
/**
 * {@inheritDoc}
 */

```

```

@Override
public double value(double x, double y) {
    if (x < 0 || x > 1) {
        throw new OutOfRangeException(x, 0, 1);
    }
    if (y < 0 || y > 1) {
        throw new OutOfRangeException(y, 0, 1);
    }
    final double x2 = x * x;
    final double x3 = x2 * x;
    final double[] pX = { 1, x, x2, x3 };
    final double y2 = y * y;
    final double y3 = y2 * y;
    final double[] pY = { 1, y, y2, y3 };
    return apply(pX, 0, pY, 0, a);
}

/**
 * Compute the value of the bicubic polynomial.
 *
 * <p>
 * Assumes the powers are zero below the provided index, and 1 at the
provided
 * index. This allows skipping some zero products and optimising multi
plication
 * by one.
 *
 * @param pX    Powers of the x-coordinate.
 * @param i      Index of pX[i] == 1
 * @param pY    Powers of the y-coordinate.
 * @param j      Index of pY[j] == 1
 * @param coeff Spline coefficients.
 * @return the interpolated value.
 */
private static double apply(double[] pX, int i, double[] pY, int j, do
uble[][] coeff) {
    // assert pX[i] == 1
    double result = sumOfProducts(coeff[i], pY, j);
    while (++i < N) {
        final double r = sumOfProducts(coeff[i], pY, j);
        result += r * pX[i];
    }
    return result;
}

/**

```

```

    * Compute the sum of products starting from the provided index. Assumes that
    * factor {@code b[j] == 1}.
    *
    * @param a Factors.
    * @param b Factors.
    * @param j Index to initialise the sum.
    * @return the double
    */
private static double sumOfProducts(double[] a, double[] b, int j) {
    // assert b[j] == 1
    final Sum sum = Sum.of(a[j]);
    while (++j < N) {
        sum.addProduct(a[j], b[j]);
    }
    return sum.getAsDouble();
}
/**
 * @return the partial derivative wrt {@code x}.
 */
BivariateFunction partialDerivativeX() {
    return partialDerivativeX;
}
/**
 * @return the partial derivative wrt {@code y}.
 */
BivariateFunction partialDerivativeY() {
    return partialDerivativeY;
}
/**
 * @return the second partial derivative wrt {@code x}.
 */
BivariateFunction partialDerivativeXX() {
    return partialDerivativeXX;
}
/**
 * @return the second partial derivative wrt {@code y}.
 */
BivariateFunction partialDerivativeYY() {
    return partialDerivativeYY;
}
/**
 * @return the second partial cross-derivative.
 */

```

```

BivariateFunction partialDerivativeXY() {
    return partialDerivativeXY;
}
}
class Bicubic_VT {
    /* ----- helpers ----- */
    private static void headline(String name) {
        System.out.println("\n==== " + name + " ====");
    }
    @FunctionalInterface
    private interface Throwing {
        void run() throws Exception;
    }
    private static void expectThrow(String name, Throwing r, Class<?> expected) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expected.getSimpleName());
        } catch (java.lang.reflect.InvocationTargetException ite) {
            Throwable t = ite.getTargetException();
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
            System.out.printf("[%s] %s threw expected\n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
            System.out.printf("[%s] %s threw expected\n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
        }
    }
    private static String fmt(double v) {
        if (Double.isNaN(v))
            return "NaN";
        if (v == Double.POSITIVE_INFINITY)
            return "+Inf";
        if (v == Double.NEGATIVE_INFINITY)
            return "-Inf";
        if (v == 0.0)
            return (Double.doubleToRawLongBits(v) >>> 63) == 1 ? "-0.0" : "0.0";
        return String.format(java.util.Locale.ROOT, "%.17g", v);
    }
}

```

```

private static void checkEquals(String name, double expected, double actual, double relTol) {
    boolean ok;
    if (Double.isNaN(expected) || Double.isNaN(actual)) {
        ok = (Double.isNaN(expected) && Double.isNaN(actual));
    } else if (Double.isInfinite(expected) || Double.isInfinite(actual)) {
        ok = (Double.doubleToRawLongBits(expected) == Double.doubleToRawLongBits(actual));
    } else {
        double tol = Math.max(1.0, Math.max(Math.abs(expected), Math.abs(actual))) * relTol;
        ok = (expected == actual) || (Math.abs(expected - actual) <= tol);
    }
    System.out.printf("[%s] %s expected=%s actual=%s (relTol=%.1e)%n",
        ok ? "PASS" : "FAIL", name, fmt(expected),
        fmt(actual), relTol);
}

/* ----- grid builders for f(x,y)=x+2y ----- */
private static double[][] new2D(int nx, int ny) {
    return new double[nx][ny];
}

private static Object[] buildLinearGrid(double[] x, double[] y) {
    final int nx = x.length, ny = y.length;
    double[][] f = new2D(nx, ny);
    double[][] fx = new2D(nx, ny);
    double[][] fy = new2D(nx, ny);
    double[][] fxy = new2D(nx, ny);
    for (int i = 0; i < nx; i++) {
        for (int j = 0; j < ny; j++) {
            f[i][j] = x[i] + 2.0 * y[j];
            fx[i][j] = 1.0;
            fy[i][j] = 2.0;
            fxy[i][j] = 0.0;
        }
    }
    return new Object[] { f, fx, fy, fxy };
}

/* ----- reflection helpers to hit inner polynomial ----- */
private static double[][] makeInnerCoeffs() {
    double[][] a = new double[4][4];
    a[0][0] = 1.0;
    a[1][0] = 2.0;
    a[0][1] = 3.0;

```

```

        a[1][1] = 4.0;
        a[2][0] = 0.5;
        a[0][2] = -0.25;
        a[3][0] = 0.1;
        a[0][3] = -0.05;
        return a;
    }

    private static Object constructInnerPolynomialFlex(Object outerInstance) throws Exception {
        final Class<?> outer = BicubicInterpolatingFunction.class;
        final java.util.List<Class<?>> innerList = java.util.Arrays.asList(outer.getDeclaredClasses());
        for (Class<?> in : innerList) {
            for (java.lang.reflect.Constructor<?> c : in.getDeclaredConstructors()) {
                final Class<?>[] pt = c.getParameterTypes();
                if (pt.length == 0)
                    continue;
                boolean hasBool = false, has2D = false;
                for (Class<?> t : pt) {
                    if (t == boolean.class)
                        hasBool = true;
                    if (t.isArray() && t.getComponentType() != null && t.getComponentType().isArray()
                        && t.getComponentType().getComponentType() == double.class) {
                        has2D = true;
                    }
                }
                if (!hasBool || !has2D)
                    continue;
                final Object[] args = new Object[pt.length];
                boolean ok = true;
                for (int i = 0; i < pt.length; i++) {
                    final Class<?> t = pt[i];
                    if (t == outer) {
                        if (outerInstance == null) {
                            ok = false;
                            break;
                        }
                        args[i] = outerInstance;
                    } else if (t.isArray() && t.getComponentType() != null && t.getComponentType().isArray()
                        && t.getComponentType().getComponentType() == double.class) {
                        args[i] = makeInnerCoeffs();
                    }
                }
            }
        }
    }

```

```

    } else if (t.isArray() && t.getComponentType() == double.class) {
        args[i] = new double[] { 1.0, 0.0, 0.0, 0.0 };
    } else if (t == double.class) {
        args[i] = 1.0;
    } else if (t == int.class) {
        args[i] = 4; // N=4
    } else if (t == boolean.class) {
        args[i] = Boolean.TRUE;
    } else {
        ok = false;
        break;
    }
}
if (!ok)
    continue;
try {
    c.setAccessible(true);
    return c.newInstance(args);
} catch (Throwable ignore) {
    /* try next */ }
}
}
throw new IllegalStateException("No inner polynomial ctor matched (needs boolean & double[][]).");
}
private static Object safeConstructInnerPolynomial(Object outerInstance) {
    try {
        return constructInnerPolynomialFlex(outerInstance);
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
private static Object callInnerValue(Object poly, double x, double y)
throws Exception {
    for (java.lang.reflect.Method m : poly.getClass().getDeclaredMethods()) {
        if (m.getName().equals("value")) {
            Class<?>[] pt = m.getParameterTypes();
            if (pt.length == 2 && pt[0] == double.class && pt[1] == double.class) {
                m.setAccessible(true);
                return m.invoke(poly, x, y);
            }
        }
    }
}

```

```

    }
}
    throw new NoSuchMethodException("value(double,double) not found on inner polynomial");
}
    private static BicubicInterpolatingFunction constructOuterWithDerivs(double[] x, double[] y, double[][] f,
        double[][] fx, double[][] fy, double[][] fxy) {
        final Class<?> outer = BicubicInterpolatingFunction.class;
        java.lang.reflect.Constructor<?> fallback = null;
        for (java.lang.reflect.Constructor<?> c : outer.getDeclaredConstructors()) {
            final Class<?>[] pt = c.getParameterTypes();
            boolean hasBool = false;
            for (Class<?> t : pt)
                if (t == boolean.class) {
                    hasBool = true;
                    break;
                }
            if (!hasBool) {
                if (fallback == null)
                    fallback = c;
                continue;
            }
            Object[] args = new Object[pt.length];
            int used2D = 0;
            boolean ok = true;
            for (int i = 0; i < pt.length; i++) {
                Class<?> t = pt[i];
                if (t.isArray() && t.getComponentType() == double.class) {
                    args[i] = (used2D == -1) ? y : x;
                    used2D = (used2D == -1) ? -2 : -1;
                } else if (t.isArray() && t.getComponentType() != null && t.getComponentType().isArray()
                    && t.getComponentType().getComponentType() == double.class) {
                    // double[][]
                    if (used2D == 0)
                        args[i] = f;
                    else if (used2D == 1)
                        args[i] = fx;
                    else if (used2D == 2)
                        args[i] = fy;
                    else if (used2D == 3)
                        args[i] = fxy;

```

```

        else {
            ok = false;
            break;
        }
        used2D++;
    } else if (t == boolean.class) {
        args[i] = java.lang.Boolean.TRUE;
    } else if (t == int.class) {
        args[i] = 4;
    } else if (t == double.class) {
        args[i] = 1.0;
    } else {
        ok = false;
        break;
    }
}
if (!ok)
    continue;
try {
    c.setAccessible(true);
    return (BicubicInterpolatingFunction) c.newInstance(args);
} catch (Throwable ignore) {
}
}
if (fallback != null) {
    try {
        final Class<?>[] pt = fallback.getParameterTypes();
        Object[] args = new Object[pt.length];
        int used2D = 0, used1D = 0;
        for (int i = 0; i < pt.length; i++) {
            Class<?> t = pt[i];
            if (t.isArray() && t.getComponentType() == double.class) {
                args[i] = (used1D == 0) ? x : y;
                used1D++;
            } else if (t.isArray() && t.getComponentType() != null && t.getComponentType().isArray()
                && t.getComponentType().getComponentType() == double.class) {
                if (used2D == 0)
                    args[i] = f;
                else if (used2D == 1)
                    args[i] = fx;
                else if (used2D == 2)
                    args[i] = fy;
                else if (used2D == 3)

```

```

        args[i] = fxy;
        used2D++;
    } else if (t == int.class) {
        args[i] = 4;
    } else if (t == double.class) {
        args[i] = 1.0;
    } else if (t == boolean.class) {
        args[i] = java.lang.Boolean.TRUE;
    } else {
        args[i] = null;
    }
}
fallback.setAccessible(true);
return (BicubicInterpolatingFunction) fallback.newInstance(args);
} catch (Throwable e) {
    throw new RuntimeException("Fallback outer ctor failed", e);
}
}
throw new RuntimeException("No usable BicubicInterpolatingFunction constructor found.");
}
/* ----- V-method tests (merged 20 cases) ----- */
public void Vtest() {
    // ===== Constructor guards =====
    // VT01
    expectThrow("VT01 x=null",
        () -> new BicubicInterpolatingFunction(null, new double[] { 0, 1 },
            new double[][] { { 0, 0 }, { 0, 0 } }, new double[][] { { 0, 0 },
{ 0, 0 } },
            new double[][] { { 0, 0 }, { 0, 0 } }, new double[][] { { 0, 0 },
{ 0, 0 } })),
        NullArgumentException.class);
    // VT02
    expectThrow("VT02 y=null",
        () -> new BicubicInterpolatingFunction(new double[] { 0, 1 }, null,
            new double[][] { { 0, 0 }, { 0, 0 } }, new double[][] { { 0, 0 },
{ 0, 0 } },
            new double[][] { { 0, 0 }, { 0, 0 } }, new double[][] { { 0, 0 },
{ 0, 0 } })),
        NullArgumentException.class);
    // VT03
    expectThrow("VT03 f=null",
        () -> new BicubicInterpolatingFunction(new double[] { 0, 1 }, new d
ouble[] { 0, 1 }, null,

```

```

        new double[][] { { 0, 0 }, { 0, 0 } }, new double[][] { { 0, 0 },
{ 0, 0 } },
        new double[][] { { 0, 0 }, { 0, 0 } })),
        NullPointerException.class);
// VT04
expectThrow(
    "VT04 |x|=0", () -> new BicubicInterpolatingFunction(new double[] {
}, new double[] { 0, 1 },
        new double[][] {}, new double[][] {}, new double[][] {}, new doub
le[][] {}),
        NoDataException.class);
// VT05
expectThrow("VT05 |x|<2",
    () -> new BicubicInterpolatingFunction(new double[] { 0 }, new doub
le[] { 0, 1 },
        new double[][] { { 0, 0 } }, new double[][] { { 0, 0 } }, new dou
ble[][] { { 0, 0 } },
        new double[][] { { 0, 0 } })),
        NumberIsTooSmallException.class);
// VT06
expectThrow("VT06 |y|<2",
    () -> new BicubicInterpolatingFunction(new double[] { 0, 1 }, new d
ouble[] { 0 },
        new double[][] { { 0 }, { 0 } }, new double[][] { { 0 }, { 0 } },
        new double[][] { { 0 }, { 0 } }, new double[][] { { 0 }, { 0 } })),
        NumberIsTooSmallException.class);
// VT07
double[] x2 = { 0, 1 }, y2 = { 0, 1 };
double[][] f_rows3 = new double[3][2];
expectThrow("VT07 rows mismatch", () -> new BicubicInterpolatingFunct
ion(x2, y2, f_rows3, new double[2][2],
        new double[2][2], new double[2][2]), DimensionMismatchException.cla
ss);
// VT08
double[][] f_bad_inner = { new double[] { 0, 0 }, new double[] { 0, 0
, 0 } };
expectThrow("VT08 inner cols mismatch", () -> new BicubicInterpolatin
gFunction(x2, y2, f_bad_inner,
        new double[2][2], new double[2][2], new double[2][2]), DimensionMis
matchException.class);
// VT09
double[] x_nonmono = { 0, 0, 2 };
double[] y_okM = { 0, 1, 2 };

```

```

expectThrow("VT09 x non-monotone", () -> {
    Object[] g = buildLinearGrid(x_nonmono, y_okM);
    new BicubicInterpolatingFunction(x_nonmono, y_okM, (double[][][]) g[0]
, (double[][][]) g[1], (double[][][]) g[2],
    (double[][][]) g[3]);
}, NonMonotonicSequenceException.class);
// VT10
double[] y_nonmono = { 0, 2, 1 };
double[] x_okM = { 0, 1, 2 };
expectThrow("VT10 y non-monotone", () -> {
    Object[] g = buildLinearGrid(x_okM, y_nonmono);
    new BicubicInterpolatingFunction(x_okM, y_nonmono, (double[][][]) g[0]
, (double[][][]) g[1], (double[][][]) g[2],
    (double[][][]) g[3]);
}, NonMonotonicSequenceException.class);
// VT11
double[] x_withNaN = { 0, java.lang.Double.NaN, 2 };
expectThrow("VT11 x has NaN", () -> {
    Object[] g = buildLinearGrid(x_withNaN, y_okM);
    new BicubicInterpolatingFunction(x_withNaN, y_okM, (double[][][]) g[0]
, (double[][][]) g[1], (double[][][]) g[2],
    (double[][][]) g[3]);
}, NotFiniteNumberException.class);
// VT12
headline("VT12 f has NaN");
{
    double[] x3 = { 0, 1, 2 };
    double[] y3 = { 0, 1, 3 };
    Object[] g = buildLinearGrid(x3, y3);
    double[][] f = (double[][][]) g[0];
    f[1][2] = java.lang.Double.NaN;
    expectThrow("VT12 f NaN", () -> new BicubicInterpolatingFunction(x3,
y3, f, (double[][][]) g[1],
    (double[][][]) g[2], (double[][][]) g[3]), NotFiniteNumberException.cl
ass);
}
// VT13
headline("VT13 fx negative infinity");
{
    double[] x3 = { 0, 1, 2 };
    double[] y3 = { 0, 1, 3 };
    Object[] g = buildLinearGrid(x3, y3);
    double[][] fx = (double[][][]) g[1];
    fx[0][0] = java.lang.Double.NEGATIVE_INFINITY;
}

```

```

        expectThrow("VT13 fx -
Inf", () -> new BicubicInterpolatingFunction(x3, y3, (double[][][]) g[0],
fx,
        (double[][][]) g[2], (double[][][]) g[3]), NotFiniteNumberException.cl
ass);
    }
    // ===== Construct OK + value guards =====
    double[] X = { 0, 1, 2 };
    double[] Y = { 0, 1, 3 };
    Object[] G = buildLinearGrid(X, Y);
    BicubicInterpolatingFunction B = new BicubicInterpolatingFunction(X,
Y, (double[][][]) G[0], (double[][][]) G[1],
        (double[][][]) G[2], (double[][][]) G[3]);
    // VT14
    headline("VT14 interior value correctness");
    double y14 = B.value(0.3, 0.7);
    checkEquals("VT14 value", 0.3 + 2.0 * 0.7, y14, 2e-12);
    // VT15
    expectThrow("VT15 X=NaN", () -> B.value(java.lang.Double.NaN, 0.5), N
otFiniteNumberException.class);
    expectThrow("VT15b Y=+Inf", () -> B.value(0.5, java.lang.Double.POSIT
IVE_INFINITY),
        NotFiniteNumberException.class);
    // VT16
    expectThrow("VT16 X<a", () -> B.value(-1e-
12, 0.5), OutOfRangeException.class);
    // VT17
    headline("VT17 boundary clamp X==last");
    double y17 = B.value(2.0, 0.25);
    checkEquals("VT17 value", 2.0 + 2.0 * 0.25, y17, 1e-12);
    double[] Xd = { 0, 1, 2 };
    double[] Yd = { 0, 1, 3 };
    Object[] Gd = buildLinearGrid(Xd, Yd);
    double[][] Fd = (double[][][]) Gd[0];
    double[][] FXd = (double[][][]) Gd[1];
    double[][] FYd = (double[][][]) Gd[2];
    double[][] FXYd = (double[][][]) Gd[3];
    // VT18:
    headline("VT18 construct outer with derivatives=true");
    BicubicInterpolatingFunction B2 = constructOuterWithDerivs(Xd, Yd, Fd
, FXd, FYd, FXYd);
    double v18 = B2.value(0.3, 0.7);
    checkEquals("VT18 value", 0.3 + 2.0 * 0.7, v18, 2e-12);
    // VT19:

```

```

headline("VT19 compare boundary");
double v19 = B2.value(2.0, 0.25);
checkEquals("VT19 boundary value", 2.0 + 2.0 * 0.25, v19, 1e-12);
// VT20:
headline("VT20 NaN guard on outer constructed-with-derivs");
expectThrow("VT20 X=NaN", () -> B2.value(java.lang.Double.NaN, 0.5),
NotFiniteNumberException.class);
}
}
//FT
class Bicubic_FT {
    /* ----- helpers ----- */
    private static void headline(String name) {
        System.out.println("\n==== " + name + " ====");
    }
    private static void expectThrow(String name, Runnable r, Class<?> expected) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expected.getSimpleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
            System.out.printf("[%s] %s threw expected\n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
        }
    }
    private static void assertFinite(String name, double v) {
        boolean ok = !(Double.isNaN(v) || Double.isInfinite(v));
        System.out.printf("[%s] %s -> %s\n", ok ? "PASS" : "FAIL", name,
            ok ? String.format(java.util.Locale.ROOT, "%.17g", v) : "non-finite");
    }
    private static double[] randMonotone(java.util.Random r, int n, double base, double stepMin, double stepMax) {
        double[] a = new double[n];
        double cur = base;
        for (int i = 0; i < n; i++) {
            double step = stepMin + (stepMax - stepMin) * r.nextDouble();
            cur += step;
            a[i] = cur;
        }
    }
}

```

```

    return a;
}
/** 随机多项
式  $f(x,y)=a_{00} + a_{10} x + a_{01} y + a_{20} x^2 + a_{02} y^2 + a_{11} xy$  */
private static double evalF(double x, double y, double[] c) {
    return c[0] + c[1] * x + c[2] * y + c[3] * x * x + c[4] * y * y + c[5]
] * x * y;
}
private static double dFx(double x, double y, double[] c) {
    return c[1] + 2 * c[3] * x + c[5] * y;
}
private static double dFy(double x, double y, double[] c) {
    return c[2] + 2 * c[4] * y + c[5] * x;
}
private static double dFxy(double[] c) {
    return c[5];
}
private static Object[] buildGridFromPoly(double[] xs, double[] ys, double[] coeff) {
    int nx = xs.length, ny = ys.length;
    double[][] f = new double[nx][ny];
    double[][] fx = new double[nx][ny];
    double[][] fy = new double[nx][ny];
    double[][] fxy = new double[nx][ny];
    for (int i = 0; i < nx; i++) {
        for (int j = 0; j < ny; j++) {
            double x = xs[i], y = ys[j];
            f[i][j] = evalF(x, y, coeff);
            fx[i][j] = dFx(x, y, coeff);
            fy[i][j] = dFy(x, y, coeff);
            fxy[i][j] = dFxy(coeff);
        }
    }
    return new Object[] { f, fx, fy, fxy };
}
public void Ftest() {
    final java.util.Random R = new java.util.Random(20251022L);
    // ----- 成功类随机用例（有限值断言） -----
    // FT01
    headline("FT01 random interior eval");
    {
        int nx = 3 + R.nextInt(3); // 3..5
        int ny = 3 + R.nextInt(3);
        double[] xs = randMonotone(R, nx, -10.0, 0.1, 5.0);

```

```

double[] ys = randMonotone(R, ny, -2.0, 0.05, 3.0);
double[] c = new double[] { R.nextDouble(), 10 * R.nextDouble() - 5,
10 * R.nextDouble() - 5,
    R.nextDouble() - 0.5, R.nextDouble() - 0.5, 2 * R.nextDouble() - 1
};
Object[] G = buildGridFromPoly(xs, ys, c);
BicubicInterpolatingFunction B = new BicubicInterpolatingFunction(xs
, ys, (double[][]) G[0],
    (double[][]) G[1], (double[][]) G[2], (double[][]) G[3]);
double xq = xs[0] + (xs[nx - 1] - xs[0]) * R.nextDouble();
double yq = ys[0] + (ys[ny - 1] - ys[0]) * R.nextDouble();
double v = B.value(xq, yq);
assertFinite("FT01 value", v);
}
// FT02
headline("FT02 interior near left-bottom");
{
    int nx = 4, ny = 4;
    double[] xs = randMonotone(R, nx, -1.0, 1e-9, 1e-6);
    double[] ys = randMonotone(R, ny, 2.0, 1e-9, 1e-6);
    double[] c = new double[] { 1, 2, 3, 0.01, -0.02, 0.005 };
    Object[] G = buildGridFromPoly(xs, ys, c);
    BicubicInterpolatingFunction B = new BicubicInterpolatingFunction(xs
, ys, (double[][]) G[0],
        (double[][]) G[1], (double[][]) G[2], (double[][]) G[3]);
    double v = B.value(xs[0] + 1e-12 * (xs[1] - xs[0]), ys[0] + 1e-
12 * (ys[1] - ys[0]));
    assertFinite("FT02 value", v);
}
// FT03
headline("FT03 interior near right-top");
{
    int nx = 5, ny = 3;
    double[] xs = randMonotone(R, nx, 1.0, 1e-3, 1.0);
    double[] ys = randMonotone(R, ny, 5.0, 1e-3, 1.0);
    double[] c = new double[] { -3, 4, -2, 0.2, 0.1, -0.3 };
    Object[] G = buildGridFromPoly(xs, ys, c);
    BicubicInterpolatingFunction B = new BicubicInterpolatingFunction(xs
, ys, (double[][]) G[0],
        (double[][]) G[1], (double[][]) G[2], (double[][]) G[3]);
    double v = B.value(xs[nx - 2] + 0.999 * (xs[nx - 1] - xs[nx - 2]),
        ys[ny - 2] + 0.999 * (ys[ny - 1] - ys[ny - 2]));
    assertFinite("FT03 value", v);
}

```

```

// FT04
headline("FT04 center of random middle cell");
{
    int nx = 6, ny = 6;
    double[] xs = randMonotone(R, nx, -100.0, 1e-2, 10.0);
    double[] ys = randMonotone(R, ny, 200.0, 1e-2, 10.0);
    double[] c = new double[] { R.nextDouble(), R.nextGaussian(), R.next
Gaussian(), 0.01 * R.nextGaussian(),
        0.01 * R.nextGaussian(), 0.05 * R.nextGaussian() };
    Object[] G = buildGridFromPoly(xs, ys, c);
    BicubicInterpolatingFunction B = new BicubicInterpolatingFunction(xs
, ys, (double[][][]) G[0],
        (double[][][]) G[1], (double[][][]) G[2], (double[][][]) G[3]);
    int i = 1 + R.nextInt(nx - 2), j = 1 + R.nextInt(ny - 2);
    double v = B.value(0.5 * (xs[i] + xs[i + 1]), 0.5 * (ys[j] + ys[j +
1]));
    assertFinite("FT04 value", v);
}
// FT05
headline("FT05 boundary X=min");
{
    int nx = 4, ny = 5;
    double[] xs = randMonotone(R, nx, -5, 0.1, 3.0);
    double[] ys = randMonotone(R, ny, 0, 0.1, 3.0);
    double[] c = new double[] { 2, -1, 0.5, -0.03, 0.02, 0.01 };
    Object[] G = buildGridFromPoly(xs, ys, c);
    BicubicInterpolatingFunction B = new BicubicInterpolatingFunction(xs
, ys, (double[][][]) G[0],
        (double[][][]) G[1], (double[][][]) G[2], (double[][][]) G[3]);
    double v = B.value(xs[0], ys[0] + (ys[ny - 1] - ys[0]) * R.nextDoubl
e());
    assertFinite("FT05 value", v);
}
// FT06
headline("FT06 boundary X=max");
{
    int nx = 4, ny = 5;
    double[] xs = randMonotone(R, nx, 2, 0.1, 3.0);
    double[] ys = randMonotone(R, ny, 1, 0.1, 3.0);
    double[] c = new double[] { -1, 2, 1, 0.02, -0.01, 0.03 };
    Object[] G = buildGridFromPoly(xs, ys, c);
    BicubicInterpolatingFunction B = new BicubicInterpolatingFunction(xs
, ys, (double[][][]) G[0],
        (double[][][]) G[1], (double[][][]) G[2], (double[][][]) G[3]);

```

```

    double v = B.value(xs[nx - 1], ys[0] + (ys[ny - 1] - ys[0]) * R.next
Double());
    assertFinite("FT06 value", v);
}
// FT07
headline("FT07 boundary Y=min");
{
    int nx = 3, ny = 6;
    double[] xs = randMonotone(R, nx, -2, 0.01, 1.0);
    double[] ys = randMonotone(R, ny, 10, 0.01, 1.0);
    double[] c = new double[] { 1, -4, 3, 0.04, 0.03, -0.02 };
    Object[] G = buildGridFromPoly(xs, ys, c);
    BicubicInterpolatingFunction B = new BicubicInterpolatingFunction(xs
, ys, (double[][][]) G[0],
    (double[][][]) G[1], (double[][][]) G[2], (double[][][]) G[3]);
    double v = B.value(xs[0] + (xs[nx - 1] - xs[0]) * R.nextDouble(), ys
[0]);
    assertFinite("FT07 value", v);
}
// FT08
headline("FT08 boundary Y=max");
{
    int nx = 5, ny = 5;
    double[] xs = randMonotone(R, nx, -3, 0.1, 2.0);
    double[] ys = randMonotone(R, ny, 20, 0.1, 2.0);
    double[] c = new double[] { 0.1, 0.2, -0.3, 0.0, 0.0, 0.05 };
    Object[] G = buildGridFromPoly(xs, ys, c);
    BicubicInterpolatingFunction B = new BicubicInterpolatingFunction(xs
, ys, (double[][][]) G[0],
    (double[][][]) G[1], (double[][][]) G[2], (double[][][]) G[3]);
    double v = B.value(xs[0] + (xs[nx - 1] - xs[0]) * R.nextDouble(), ys
[ny - 1]);
    assertFinite("FT08 value", v);
}
// FT09
headline("FT09 wide ranges");
{
    int nx = 6, ny = 6;
    double[] xs = randMonotone(R, nx, -1e9, 1e3, 1e7);
    double[] ys = randMonotone(R, ny, 1e12, 1e6, 1e10);
    double[] c = new double[] { R.nextGaussian() * 1e2, R.nextGaussian()
* 1e-2, R.nextGaussian() * 1e-2,
    R.nextGaussian() * 1e-6, R.nextGaussian() * 1e-
6, R.nextGaussian() * 1e-6 };

```

```

    Object[] G = buildGridFromPoly(xs, ys, c);
    BicubicInterpolatingFunction B = new BicubicInterpolatingFunction(xs
, ys, (double[][][]) G[0],
    (double[][][]) G[1], (double[][][]) G[2], (double[][][]) G[3]);
    double v = B.value(xs[2] + 0.333 * (xs[3] - xs[2]), ys[3] + 0.666 *
(ys[4] - ys[3]));
    assertFinite("FT09 value", v);
}
// FT10
headline("FT10 tiny ranges");
{
    int nx = 4, ny = 4;
    double[] xs = randMonotone(R, nx, 0.0, 1e-15, 1e-12);
    double[] ys = randMonotone(R, ny, 0.0, 1e-15, 1e-12);
    double[] c = new double[] { 1, 1, 1, 1e-6, -1e-6, 1e-6 };
    Object[] G = buildGridFromPoly(xs, ys, c);
    BicubicInterpolatingFunction B = new BicubicInterpolatingFunction(xs
, ys, (double[][][]) G[0],
    (double[][][]) G[1], (double[][][]) G[2], (double[][][]) G[3]);
    double v = B.value(0.5 * (xs[1] + xs[2]), 0.5 * (ys[1] + ys[2]));
    assertFinite("FT10 value", v);
}
// ----- 异常与边界类随机用例 -----
// FT11: X < min
headline("FT11 out-of-range X<a");
{
    double[] xs = new double[] { 0, 1, 2 };
    double[] ys = new double[] { -1, 0, 1 };
    double[] c = new double[] { 0, 1, 2, 0.1, -0.1, 0.05 };
    Object[] G = buildGridFromPoly(xs, ys, c);
    BicubicInterpolatingFunction B = new BicubicInterpolatingFunction(xs
, ys, (double[][][]) G[0],
    (double[][][]) G[1], (double[][][]) G[2], (double[][][]) G[3]);
    expectThrow("FT11", () -> B.value(-
0.5, 0.0), OutOfRangeException.class);
}
// FT12: X > max
headline("FT12 out-of-range X>b");
{
    double[] xs = new double[] { -2, -1, 0 };
    double[] ys = new double[] { 5, 6, 7, 9 };
    double[] c = new double[] { 1, -1, 0.5, -0.02, 0.03, 0.01 };
    Object[] G = buildGridFromPoly(xs, ys, c);

```

```

        BicubicInterpolatingFunction B = new BicubicInterpolatingFunction(xs
, ys, (double[][][]) G[0],
        (double[][][]) G[1], (double[][][]) G[2], (double[][][]) G[3]);
        expectThrow("FT12", () -> B.value(0.1, 6.1), OutOfRangeException.class);
    }
    // FT13: Y < min
    headline("FT13 out-of-range Y<a");
    {
        double[] xs = new double[] { -3, -2, -1, 0 };
        double[] ys = new double[] { 2, 3, 4 };
        double[] c = new double[] { 0, 2, -3, 0.02, -0.01, 0.0 };
        Object[] G = buildGridFromPoly(xs, ys, c);
        BicubicInterpolatingFunction B = new BicubicInterpolatingFunction(xs
, ys, (double[][][]) G[0],
        (double[][][]) G[1], (double[][][]) G[2], (double[][][]) G[3]);
        expectThrow("FT13", () -> B.value(-
2.5, 1.999999999), OutOfRangeException.class);
    }
    // FT14: Y > max
    headline("FT14 out-of-range Y>b");
    {
        double[] xs = new double[] { 10, 20, 30 };
        double[] ys = new double[] { -5, 0, 5 };
        double[] c = new double[] { 3, -2, 1, 0.0, 0.0, 0.03 };
        Object[] G = buildGridFromPoly(xs, ys, c);
        BicubicInterpolatingFunction B = new BicubicInterpolatingFunction(xs
, ys, (double[][][]) G[0],
        (double[][][]) G[1], (double[][][]) G[2], (double[][][]) G[3]);
        expectThrow("FT14", () -> B.value(15.0, 6.0), OutOfRangeException.class);
    }
    // FT15: X=NaN
    headline("FT15 X=NaN");
    {
        double[] xs = new double[] { 0, 1, 2 };
        double[] ys = new double[] { 3, 4, 5 };
        double[] c = new double[] { 0.1, 0.2, 0.3, 0.0, 0.0, 0.0 };
        Object[] G = buildGridFromPoly(xs, ys, c);
        BicubicInterpolatingFunction B = new BicubicInterpolatingFunction(xs
, ys, (double[][][]) G[0],
        (double[][][]) G[1], (double[][][]) G[2], (double[][][]) G[3]);
        expectThrow("FT15", () -> B.value(java.lang.Double.NaN, 3.5), NotFiniteNumberException.class);
    }

```

```

}
// FT16: Y=+Inf
headline("FT16 Y=+Inf");
{
    double[] xs = new double[] { -1, 0, 1 };
    double[] ys = new double[] { -2, 0, 2 };
    double[] c = new double[] { -1, 2, -3, 0.05, -0.04, 0.01 };
    Object[] G = buildGridFromPoly(xs, ys, c);
    BicubicInterpolatingFunction B = new BicubicInterpolatingFunction(xs
, ys, (double[][] G[0],
    (double[][] G[1], (double[][] G[2], (double[][] G[3]));
    expectThrow("FT16", () -> B.value(0.0, java.lang.Double.POSITIVE_INF
INITY), NotFiniteNumberException.class);
}
// FT17: f 中随机注入 NaN -> 构造期 NotFiniteNumberException
headline("FT17 inject NaN in f");
{
    double[] xs = new double[] { 0, 0.1, 0.2 };
    double[] ys = new double[] { 1, 1.2, 1.3 };
    double[] c = new double[] { 1, 1, 1, 0.0, 0.0, 0.0 };
    Object[] G = buildGridFromPoly(xs, ys, c);
    double[][] f = (double[][] G[0];
    f[R.nextInt(3)][R.nextInt(3)] = java.lang.Double.NaN;
    expectThrow("FT17", () -> new BicubicInterpolatingFunction(xs, ys, f
, (double[][] G[1], (double[][] G[2],
    (double[][] G[3]), NotFiniteNumberException.class);
}
// FT18: fx 中随机注入 +Inf -> 构造期 NotFiniteNumberException
headline("FT18 inject +Inf in fx");
{
    double[] xs = new double[] { -10, -9, -8, -7 };
    double[] ys = new double[] { 100, 110, 130, 160 };
    double[] c = new double[] { 0.5, -0.5, 0.25, 0.0, 0.0, 0.0 };
    Object[] G = buildGridFromPoly(xs, ys, c);
    double[][] fx = (double[][] G[1];
    fx[1][2] = java.lang.Double.POSITIVE_INFINITY;
    expectThrow("FT18", () -> new BicubicInterpolatingFunction(xs, ys, (
double[][] G[0], fx, (double[][] G[2],
    (double[][] G[3]), NotFiniteNumberException.class);
}
// FT19: 非单调 x -> NonMonotonicSequenceException
headline("FT19 non-monotone x");
{
    double[] xs = new double[] { 0.0, 1.0, 0.5 }; // 非严格递增

```

```

    double[] ys = new double[] { 0.0, 1.0, 2.0 };
    double[] c = new double[] { 1, 2, 3, 0, 0, 0 };
    Object[] G = buildGridFromPoly(xs, ys, c);
    expectThrow("FT19", () -> new BicubicInterpolatingFunction(xs, ys, (
double[][][]) G[0], (double[][][]) G[1],
    (double[][][]) G[2], (double[][][]) G[3]), NonMonotonicSequenceExcepti
on.class);
}
// FT20: 极端紧凑网格 + 超大系数 (仍有限) -> 求值有限
headline("FT20 ultra-tight grid with large coeffs");
{
    int nx = 3, ny = 3;
    double[] xs = new double[] { 0.0, 1e-300, 2e-300 };
    double[] ys = new double[] { -1e-300, 0.0, 1e-300 };
    // 适度放大一次项, 避免溢出
    double[] c = new double[] { 1e150, 1e150, -1e150, 1e-50, 1e-50, 1e-
50 };
    Object[] G = buildGridFromPoly(xs, ys, c);
    BicubicInterpolatingFunction B = new BicubicInterpolatingFunction(xs
, ys, (double[][][]) G[0],
    (double[][][]) G[1], (double[][][]) G[2], (double[][][]) G[3]);
    double v = B.value(1e-300, 0.0);
    assertFinite("FT20 value", v);
}
System.out.println("\n[Done] BicubicInterpolatingFunction FT01-
FT20 executed.");
}
}
final class BIC_Z3 {
    // ----- helpers -----
    private static void headline(String name) {
        System.out.println("\n==== " + name + " ===");
    }
    private static void note(String s) {
        System.out.println(s);
    }
    private static void expectThrow(String name, Runnable r, Class<?> expe
cted) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expected.getS
impleName());
        } catch (Throwable t) {

```

```

        System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
        System.out.printf("[%s] %s threw expected%n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
    }
}

private static void checkEquals(String name, double expected, double actual, double tol) {
    boolean ok;
    if (Double.isNaN(expected) || Double.isNaN(actual)) {
        ok = (Double.isNaN(expected) && Double.isNaN(actual));
    } else if (Double.isInfinite(expected) || Double.isInfinite(actual)) {
        ok = (Double.doubleToRawLongBits(expected) == Double.doubleToRawLongBits(actual));
    } else {
        ok = Math.abs(expected - actual) <= tol;
    }
    System.out.printf("[%s] %s expected=%.17g actual=%.17g (tol=%.1e)%n", ok ? "PASS" : "FAIL", name, expected, actual, tol);
}

//  $f(x,y)=x+2y$  的一致网格
private static Object[] buildLinearGrid(double[] x, double[] y) {
    final int nx = x.length, ny = y.length;
    double[][] f = new double[nx][ny];
    double[][] fx = new double[nx][ny];
    double[][] fy = new double[nx][ny];
    double[][] fxy = new double[nx][ny];
    for (int i = 0; i < nx; i++) {
        for (int j = 0; j < ny; j++) {
            f[i][j] = x[i] + 2.0 * y[j];
            fx[i][j] = 1.0;
            fy[i][j] = 2.0;
            fxy[i][j] = 0.0;
        }
    }
    return new Object[] { f, fx, fy, fxy };
}

// 反射拿到 partialDerivativeX()/partialDerivativeY() 返回的对象并执行 value(x,y)
private static double evalPartial(Object bicubic, String which, double x, double y) throws Exception {

```

```

    java.lang.reflect.Method m = bicubic.getClass().getDeclaredMethod(whi
ch);
    m.setAccessible(true);
    Object bf = m.invoke(bicubic); // 可能是 BivariateFunction
    java.lang.reflect.Method mv = bf.getClass().getMethod("value", double
.class, double.class);
    return ((Double) mv.invoke(bf, x, y)).doubleValue();
}
public void Z3test() {
    // 通用 OK 网格
    final double[] X = { 0, 1, 2, 4 }; // Nx=4
    final double[] Y = { 0, 1, 2, 4, 7 }; // Ny=5
    final Object[] G = buildLinearGrid(X, Y);
    final double[][] F = (double[][][]) G[0];
    final double[][] FX = (double[][][]) G[1];
    final double[][] FY = (double[][][]) G[2];
    final double[][] FXY = (double[][][]) G[3];
    // Z301 -----
    -----
    expectThrow("Z301 any null (y=null)", () -> new BicubicInterpolatingF
unction(X, null, F, FX, FY, FXY),
        NullArgumentException.class);
    // Z302 -----
    -----
    expectThrow("Z302 |x|=0 -> NoData", () -> new BicubicInterpolatingFun
ction(new double[] {}, Y,
        new double[][] {}, new double[][] {}, new double[][] {}, new double
[][] {}), NoDataException.class);
    expectThrow("Z302b |y|=0 -> NoData", () -> new BicubicInterpolatingFu
nction(X, new double[] {},
        new double[][] {}, new double[][] {}, new double[][] {}, new double
[][] {}), NoDataException.class);
    // Z303 -----
    -----
    expectThrow("Z303 |x|<2 -> NumberIsTooSmall",
        () -> new BicubicInterpolatingFunction(new double[] { 0 }, new doub
le[] { 0, 1 },
            new double[][] { { 0, 0 } }, new double[][] { { 0, 0 } }, new dou
ble[][] { { 0, 0 } },
            new double[][] { { 0, 0 } })),
        NumberIsTooSmallException.class);
    expectThrow("Z303b |y|<2 -> NumberIsTooSmall",
        () -> new BicubicInterpolatingFunction(new double[] { 0, 1 }, new d
ouble[] { 0 },

```

```

        new double[][] { { 0 }, { 0 } }, new double[][] { { 0 }, { 0 } },
        new double[][] { { 0 }, { 0 } }, new double[][] { { 0 }, { 0 } })
    ,
    NumberIsTooSmallException.class);
    // Z304 -----
    -----
    double[][] f_rows3 = new double[3][Y.length]; // rows!=Nx
    expectThrow("Z304 dimension mismatch (rows)", () -> new BicubicInterp
olatingFunction(X, Y, f_rows3,
        new double[4][5], new double[4][5], new double[4][5]), DimensionMis
matchException.class);
    // Z305 -----
    -----
    double[] x_nonmono = { 0, 0, 2, 4 };
    expectThrow("Z305 x not strictly increasing", () -> {
        Object[] g = buildLinearGrid(x_nonmono, Y);
        new BicubicInterpolatingFunction(x_nonmono, Y, (double[][] g[0], (d
ouble[][] g[1], (double[][] g[2],
            (double[][] g[3]));
    }, NonMonotonicSequenceException.class);
    // Z306 -----
    -----
    double[] y_nonmono = { 0, 2, 1, 4, 7 };
    expectThrow("Z306 y not strictly increasing", () -> {
        Object[] g = buildLinearGrid(X, y_nonmono);
        new BicubicInterpolatingFunction(X, y_nonmono, (double[][] g[0], (d
ouble[][] g[1], (double[][] g[2],
            (double[][] g[3]));
    }, NonMonotonicSequenceException.class);
    // Z307 -----
    -----
    double[] x_withNaN = { 0, Double.NaN, 2, 4 };
    expectThrow("Z307 non-finite x/y", () -> {
        Object[] g = buildLinearGrid(x_withNaN, Y);
        new BicubicInterpolatingFunction(x_withNaN, Y, (double[][] g[0], (d
ouble[][] g[1], (double[][] g[2],
            (double[][] g[3]));
    }, NotFiniteNumberException.class);
    // Z308 -----
    -----
    headline("Z308 non-finite table entries (fx NaN)");
    {
        double[][] fxBad = deepCopy(FX);
        fxBad[1][3] = Double.NaN;
    }

```

```

    expectThrow("Z308 fx has NaN", () -> new BicubicInterpolatingFunction(X, Y, F, fxBad, FY, FXY),
        NotFiniteNumberException.class);
}
// Z309 -----
-----
headline("Z309 construct OK");
BicubicInterpolatingFunction B = new BicubicInterpolatingFunction(X,
Y, F, FX, FY, FXY);
note("[PASS] constructed OK (ex=None).");
// Z310 -----
-----
expectThrow("Z310 locate: X or Y not finite", () -> B.value(Double.NaN,
0.5 * (Y[1] + Y[2])),
    NotFiniteNumberException.class);
// Z311 -----
-----
expectThrow("Z311 locate: (X,Y) out of grid (X<a)", () -> B.value(X[0]
- 1e-9, 0.5 * (Y[1] + Y[2])),
    OutOfRangeException.class);
// Z312 -----
-----
headline("Z312 locate: right edge X==x[Nx-1]");
{
    double ymid = 0.5 * (Y[2] + Y[3]);
    double got = B.value(X[X.length - 1], ymid);
    double exp = X[X.length - 1] + 2.0 * ymid;
    checkEquals("Z312 value", exp, got, 1e-12);
}
// Z313 -----
-----
headline("Z313 locate: top edge Y==y[Ny-1]");
{
    double xmid = 0.5 * (X[1] + X[2]);
    double got = B.value(xmid, Y[Y.length - 1]);
    double exp = xmid + 2.0 * Y[Y.length - 1];
    checkEquals("Z313 value", exp, got, 1e-12);
}
// Z314 -----
-----
headline("Z314 locate: corner (X==last, Y==last)");
{
    double got = B.value(X[X.length - 1], Y[Y.length - 1]);
    double exp = X[X.length - 1] + 2.0 * Y[Y.length - 1];

```

```

    checkEquals("Z314 value", exp, got, 1e-12);
}
// Z315 -----
-----
headline("Z315 locate: interior cell");
{
    double xi = 0.3 * (X[1]) + 0.7 * (X[2]); // 严格内部
    double yi = 0.4 * (Y[2]) + 0.6 * (Y[3]);
    double got = B.value(xi, yi);
    double exp = xi + 2.0 * yi;
    checkEquals("Z315 value", exp, got, 2e-12);
}
// Z316 -----
-----
expectThrow("Z316 value: propagate OutOfRange (Y>b)",
    () -> B.value(0.5 * (X[1] + X[2]), Y[Y.length - 1] + 1e-
9), OutOfRangeException.class);
// Z317 -----
-----
headline("Z317 value: any of 16 values non-
finite -> NotFiniteNumber");
{
    // 这里构造一个“仅在使用到的单元”里含 NaN 的 fx; 若实现把检查放在构造期,
    会在构造时报错, 也可接受
    double[][] fxBad = deepCopy(FX);
    // 选取会用到的单元 [1,2]×[2,3] 之一元素置 NaN
    fxBad[1][2] = Double.NaN;
    expectThrow("Z317 at evaluation", () -> {
        BicubicInterpolatingFunction Bbad = new BicubicInterpolatingFunction(X, Y, F, fxBad, FY, FXY);
        double xi = 0.5 * (X[1] + X[2]);
        double yi = 0.5 * (Y[2] + Y[3]);
        Bbad.value(xi, yi); // 按某些实现会在此检查并抛出
    }, NotFiniteNumberException.class);
}
// Z318 -----
-----
headline("Z318 value: all finite -> None");
{
    double xi = 0.25 * (X[2] - X[1]) + X[1];
    double yi = 0.75 * (Y[3] - Y[2]) + Y[2];
    double got = B.value(xi, yi);
    double exp = xi + 2.0 * yi;
    checkEquals("Z318 value", exp, got, 2e-12);
}

```

```

    }
    // Z319 -----
    -----
    headline("Z319 partial X: success");
    try {
        double px = evalPartial(B, "partialDerivativeX", 0.5 * (X[1] + X[2])
, 0.5 * (Y[2] + Y[3]));
        checkEquals("Z319 d/dx", 1.0, px, 1e-9);
    } catch (Throwable t) {
        System.out.println("[WARN] partialDerivativeX() not available in thi
s build: " + t);
    }
    // Z320 -----
    -----
    headline("Z320 partial Y: success");
    try {
        double py = evalPartial(B, "partialDerivativeY", 0.5 * (X[1] + X[2])
, 0.5 * (Y[2] + Y[3]));
        checkEquals("Z320 d/dy", 2.0, py, 1e-9);
    } catch (Throwable t) {
        System.out.println("[WARN] partialDerivativeY() not available in thi
s build: " + t);
    }
    System.out.println("\n[Done] Z301-Z320 executed.");
}
private static double[][] deepCopy(double[][] a) {
    double[][] b = new double[a.length][];
    for (int i = 0; i < a.length; i++)
        b[i] = a[i].clone();
    return b;
}
}
/* ----- Launcher (optional) ----- */
final class BicubicVT_Runner {
    public static void main(String[] args) {
        // VT
        new Bicubic_VT().Vtest();
        // FT
        // new Bicubic_FT().Ftest();
        // Z3
        // new BIC_Z3().Z3test();
    }
}

```

- Coverage report

1. Test case number: 20
2. V-Method testing branch coverage: 62.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ BicubicInterpolatingFunction.java	20.1 %	57	227	284
> BIC_Z3	53.8 %	14	12	26
> BicubicFunction	59.1 %	13	9	22
▼ BicubicInterpolatingFunction	62.5 %	30	18	48
BicubicInterpolatingFunction(double	67.9 %	19	9	28
searchIndex(double, double[])	87.5 %	7	1	8
computeSplineCoefficients(double[])	100.0 %	4	0	4
BicubicInterpolatingFunction(double		0	0	0
partialDerivative(Function < BicubicFu		0	0	0
partialDerivativeX()		0	0	0
partialDerivativeY()		0	0	0
value(double, double)		0	0	0
> BicubicVT_Runner		0	0	0

3. V-Method testing instruction coverage: 90.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ BicubicInterpolatingFunction.java	33.8 %	2,833	5,553	8,386
> BicubicFunction	28.8 %	183	453	636
> BicubicVT_Runner	62.5 %	5	3	8
> BIC_Z3	82.5 %	1,054	223	1,277
▼ BicubicInterpolatingFunction	90.2 %	1,591	172	1,763
BicubicInterpolatingFunction(double	84.6 %	314	57	371
searchIndex(double, double[])	96.2 %	50	2	52
computeSplineCoefficients(double[])	100.0 %	39	0	39
BicubicInterpolatingFunction(double	100.0 %	10	0	10
partialDerivative(Function < BicubicFu	100.0 %	4	0	4
partialDerivativeX()	100.0 %	4	0	4
partialDerivativeY()	100.0 %	4	0	4
value(double, double)	100.0 %	58	0	58

4. Fuzz testing branch coverage: 58.3%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ commons-math4-legacy	0.3 %	53	19,948	20,001
▼ src/main/java	0.4 %	53	13,516	13,569
▼ org.apache.commons.math4.legacy.analysis.i	7.1 %	53	695	748
▼ BicubicInterpolatingFunction.java	18.7 %	53	231	284
▼ BicubicInterpolatingFunction	58.3 %	28	20	48
BicubicInterpolatingFunction(double	57.1 %	16	12	28
computeSplineCoefficients(double[])	100.0 %	4	0	4
searchIndex(double, double[])	100.0 %	8	0	8
BicubicInterpolatingFunction(double		0	0	0
value(double, double)		0	0	0
> BicubicFunction	59.1 %	13	9	22
> Bicubic_FT	75.0 %	12	4	16
> BicubicVT_Runner		0	0	0

5. Fuzz testing instruction coverage: 89.1%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ BicubicInterpolatingFunction.java	49.9 %	4,181	4,205	8,386
> BicubicFunction	28.8 %	183	453	636
> BicubicVT_Runner	62.5 %	5	3	8
▼ BicubicInterpolatingFunction	89.1 %	1,570	193	1,763
BicubicInterpolatingFunction(double	81.7 %	303	68	371
computeSplineCoefficients(double[])	100.0 %	39	0	39
searchIndex(double, double[])	100.0 %	52	0	52
BicubicInterpolatingFunction(double	100.0 %	10	0	10
value(double, double)	100.0 %	58	0	58
> Bicubic_FT	98.5 %	2,423	36	2,459

6. Z3 branch coverage: 60.4%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
> commons-math4-legacy-core	0.4 %	10	2,304	2,314
▼ commons-math4-legacy	0.5 %	103	19,898	20,001
▼ src/main/java	0.8 %	103	13,466	13,569
▼ org.apache.commons.math4.legacy.analysis.i	13.8 %	103	645	748
▼ BicubicInterpolatingFunction.java	36.3 %	103	181	284
> Bicubic_VT	35.5 %	61	111	172
> BicubicFunction	59.1 %	13	9	22
▼ BicubicInterpolatingFunction	60.4 %	29	19	48
BicubicInterpolatingFunction(double	64.3 %	18	10	28
searchIndex(double, double[])	87.5 %	7	1	8
computeSplineCoefficients(double[])	100.0 %	4	0	4
BicubicInterpolatingFunction(double		0	0	0
value(double, double)		0	0	0

7. Z3 instruction coverage: 89.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ BicubicInterpolatingFunction.java	41.1 %	3,443	4,943	8,386
> BicubicFunction	28.8 %	183	453	636
> BicubicVT_Runner	62.5 %	5	3	8
> Bicubic_VT	74.7 %	1,676	567	2,243
▼ BicubicInterpolatingFunction	89.6 %	1,579	184	1,763
BicubicInterpolatingFunction(double	84.6 %	314	57	371
searchIndex(double, double[])	96.2 %	50	2	52
computeSplineCoefficients(double[])	100.0 %	39	0	39
BicubicInterpolatingFunction(double	100.0 %	10	0	10
value(double, double)	100.0 %	58	0	58